

Course: Computer Algorithm Lab
Assignment No. 9

Amortised Analysis

1) Aggregate Analysis — Dynamic Array Expansion

In many applications (e.g., text editors, list-based inventory systems), data structures like dynamic arrays (vectors) are frequently resized when capacity is full.

You are to analyze how resizing affects the performance over time.

Implement a dynamic array that:

- Doubles its capacity when full.

- Tracks the number of element insertions.

- Record the cost of each insertion.

- Perform aggregate analysis to determine the amortized cost per insertion.

Expected Output:

Operation	Actual Cost		Total Cost	Amortized Cost
Insert 1	1	1	1	
Insert 2	2	3	1.5	
...	...	...	...	

2) Accounting Method — Stack with Multipop Operation

Consider a browser's "Back" button implementation that uses a stack.

Each operation can be:

- push(page)

- pop()

- multipop(k) (pop up to k items in one operation)

Implement stack operations with multipop(k).

Assign amortized costs using the accounting method:

- Push = 2 units (1 for push, 1 saved for pop)

- Pop = 0 units (use saved credit)

Demonstrate that the total amortized cost of n operations is $O(n)$.

3) Dynamic Tables — Table Expansion and Contraction

A mobile app keeps a frequently-changing in-memory list (e.g., a contact list or a recently-viewed cache) that must grow when it fills up and shrink when many entries are deleted, so the app does not keep holding on to memory it no longer needs.

Implement a dynamic table that:

- Doubles its size when an INSERT is attempted on a full table.

- Halves its size when a DELETE drops the table below a chosen load-factor threshold.

Build and compare two contraction policies on the same sequence of operations:

Policy A (naive): halve the table as soon as it becomes half-full (load factor 1/2).

Policy B (safe): halve the table only once it drops to one-quarter full (load factor 1/4).

Run both policies on an adversarial sequence that alternates one DELETE and one INSERT right at Policy A's resize boundary, and record the total and average cost per operation for each policy.

Expected Output (table starts at size 16, full; sequence = alternating single delete/insert at the boundary, 30 operations):

Policy	Total Cost (30 ops)	Average Cost / Operation	
A — naive (contract at 1/2 full)	285	9.50	
B — safe (contract at 1/4 full)	30	1.00	

Explain why Policy A resizes (and pays an $O(n)$ rebuild cost) on every single operation in this sequence — this is called thrashing — while Policy B's extra buffer between the expand and contract thresholds keeps every operation $O(1)$ amortized.

4) Accounting Method — Queue Using Two Stacks

A print spooler needs to serve print jobs in FIFO order, but the underlying system only exposes stack (LIFO) primitives. The spooler is built as a queue using two stacks: an inStack that receives new jobs, and an outStack that serves jobs out.

Implement a queue with:

enqueue(x): push x onto inStack.

dequeue(): if outStack is empty, move every item from inStack to outStack (reversing their order), then pop and return the top of outStack.

Assign amortized costs using the accounting method:

enqueue = 2 units (1 for the actual push, 1 saved as credit on that item for its future move to outStack).

dequeue = 1 unit (the move of each item out of inStack is paid for by that item's saved credit, not by the dequeue call that triggers it).

Demonstrate that the total amortized cost of n operations is $O(n)$, even though a single dequeue can trigger an $O(n)$ transfer in the worst case.

Expected Output (5 enqueues, then a dequeue that forces a full transfer):

Operation	Actual Cost	Amortized Charge	Credit Bank After
enqueue(1..5)	1 each	2 each	5
dequeue() →	1	6	1 0

Note that the dequeue which triggers the transfer has an actual cost of 6 (five moves plus the final pop) but is charged only 1 amortized unit — the other 5 units are drawn from the credit bank, which must never go negative at any point in the sequence.